



Activer la connexion sociale

Google · Apple — Guide pas à pas pour aller du code prêt à la production

App : SGI Mobile (Expo SDK 51) **Bundle iOS** : com.infinity-uae.sgi **Package Android** : com.infinityuae.sgi

SOMMAIRE

1. Vérifier que le code est prêt (rien à faire, déjà fait)
2. Créer les identifiants Google (Google Cloud Console)
3. Activer Sign in with Apple (Apple Developer)
4. Coller les clés dans le fichier `.env`
5. Compiler une *dev build* EAS (Expo Go ne suffit pas)
6. Tester sur ton iPhone et ton Android
7. Activer côté backend (à faire par l'équipe API)

Étape 1

Le code est déjà prêt

L'application sait déjà parler à Google et à Apple. Tu n'as **rien à coder**. Tu dois juste fournir les « clés » des deux fournisseurs pour que l'app puisse se présenter chez eux.

Pourquoi : ces clés (Client IDs / Services IDs) sont des identités numériques. Apple et Google ne te font pas confiance tant que tu n'as pas créé une identité reconnue de leur côté.

Étape 2

Créer les identifiants Google

Rends-toi sur <https://console.cloud.google.com/> et connecte-toi avec le compte Google de l'entreprise.

1. **Créer un projet** : menu en haut « Sélectionner un projet » → *Nouveau projet* → nom **SGI Mobile**.
2. **Écran de consentement OAuth** : menu de gauche → *APIs et services* → *Écran de consentement OAuth*.
 - Type d'utilisateur : **Externe**
 - Nom de l'application : **SGI – Infinity International**
 - Email du support : ton email
 - Domaine autorisé : ton domaine (ex. **infinity-uae.com**)
3. **Créer 3 OAuth Client IDs** : menu de gauche → *Identifiants* → *Créer des identifiants* → *ID client OAuth*. Recommence 3 fois :

Type	À renseigner	Variable .env
iOS	Bundle ID : com.infinity-uae.sgi	EXPO_PUBLIC_GOOGLE_IOS_CLIENT_ID
Android	Nom du package : com.infinityuae.sgi Empreinte SHA-1 : ouvre un terminal et fais eas credentials puis copie le SHA-1	EXPO_PUBLIC_GOOGLE_ANDROID_CLIENT_ID
Web	Origines JavaScript autorisées : laisser vide URI de redirection autorisée : https://auth.expo.io/@<ton-username>-expo>/sgi-mobile	EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID

Important : garde la fenêtre ouverte — chaque création te donne un long *Client ID* qui finit par **.apps.googleusercontent.com** . C'est lui qu'il faut copier.

Étape 3

Activer Sign in with Apple

Rends-toi sur <https://developer.apple.com/account/> avec ton compte Apple Developer (compte payant 99 USD/an obligatoire).

1. **App ID** : *Certificates, IDs & Profiles* → *Identifiers* → trouve [com.infinity-uae.sgi](#) (ou crée-le).

- Coche **Sign In with Apple** dans la liste des Capabilities
- Clique Save

2. **Service ID** (pour le backend) :

- Onglet *Identifiers* → + → *Services IDs*
- Description : [SGI Backend](#)
- Identifier : [com.infinity-uae.sgi.web](#)
- Coche *Sign In with Apple* → *Configure*
- Primary App ID : [com.infinity-uae.sgi](#)
- Return URLs : URL backend, ex. <https://api.sgi-uae.com/api/v1/auth/social/apple/callback>

3. **Clé privée** (pour signer côté backend) :

- Onglet *Keys* → +
- Key Name : [SGI Sign in with Apple](#)
- Coche *Sign In with Apple* → *Configure* → choisis l'App ID
- Télécharge le fichier [.p8](#) — **tu ne pourras le télécharger qu'une seule fois !**
- Note le **Key ID** (10 caractères) et ton **Team ID** (visible en haut à droite)

À transmettre au backend : le fichier [.p8](#), le Key ID, le Team ID et le Services ID. Ces 4 valeurs serviront à vérifier que les utilisateurs viennent vraiment d'Apple.

Étape 4

Coller les clés dans le fichier `.env`

Dans le dossier `apps/mobile/`, ouvre (ou crée) le fichier `.env` et complète :

```
# API
EXPO_PUBLIC_API_URL=https://api.sgi-uae.com/api/v1
EXPO_PUBLIC_APP_ENV=production

# Google OAuth – coller les 3 Client IDs récupérés à l'étape 2
EXPO_PUBLIC_GOOGLE_IOS_CLIENT_ID=1234-xxx.apps.googleusercontent.com
EXPO_PUBLIC_GOOGLE_ANDROID_CLIENT_ID=1234-yyy.apps.googleusercontent.com
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID=1234-zzz.apps.googleusercontent.com
```

Apple : rien à mettre dans `.env` côté mobile — la config Apple est dans `app.json` (`usesAppleSignIn: true` , déjà fait). Les 4 valeurs Apple (Key ID, Team ID, Services ID, fichier .p8) vont uniquement côté backend.

Étape 5

Compiler une *dev build* EAS

L'application Expo Go classique **ne peut pas** tester Sign in with Apple — c'est une limitation Apple. Il faut générer une vraie build qui s'installe sur ton téléphone.

```
# 1. Installe le CLI EAS (une seule fois)
npm install -g eas-cli

# 2. Connecte-toi à ton compte Expo
eas login

# 3. Lance la build (depuis apps/mobile/)
cd apps/mobile
eas build --profile development --platform ios
eas build --profile development --platform android
```

EAS te donne un lien — ouvre-le sur ton téléphone, installe, puis lance `npx expo start --dev-client` pour te connecter.

Astuce : la première build prend 10–20 minutes. Les suivantes sont plus rapides. Tu n'as besoin de re-build que si tu changes les *plugins* ou les *credentials natifs*.

Étape 6

Tester sur tes appareils

- ❑ Ouvrir l'app sur un iPhone — le bouton **Apple** (noir) doit apparaître
- ❑ Cliquer sur Apple → l'écran natif iOS s'ouvre → choisir email → un JWT est envoyé au backend
- ❑ Ouvrir l'app sur un Android — le bouton Apple est masqué automatiquement, Google reste visible
- ❑ Cliquer sur Google → un onglet Chrome/Safari s'ouvre → choisir compte → retour à l'app
- ❑ Vérifier que la modale de sécurité s'affiche après la connexion (rappels confidentialité)
- ❑ Vérifier les tabs : seuls les modules choisis dans « Options avancées » doivent apparaître

Si **Google échoue avec 501** : normal tant que l'étape 7 (backend) n'est pas faite. Le côté mobile remonte bien le `id_token` mais le backend ne sait pas encore le valider.

Étape 7

Activer côté backend (équipe API)

Le mobile envoie déjà `POST /api/v1/auth/social` avec `{ provider, id_token }`. Le backend retourne actuellement `501`. Pour activer :

1. **Google** : vérifier le JWT via `https://www.googleapis.com/oauth2/v3/certs` (JWKS). Audience doit matcher l'un des 3 Client IDs.
2. **Apple** : vérifier le JWT via `https://appleid.apple.com/auth/keys`. Audience = Services ID.
3. **Lookup utilisateur** : chercher par `email` issu du token. Si inconnu → créer en `status=pending` (workflow d'approbation admin existant).
4. **Émission JWT SGI** : appeler `encode_jwt(...)` comme dans le login classique, retourner `TokenResponse`.

Fichier à modifier : `apps/api/app/routers/auth/router.py`, fonction `social_login()`.

CHECKLIST FINALE

- ☐ Projet Google Cloud créé + écran de consentement configuré
 - ☐ 3 OAuth Client IDs Google (iOS, Android, Web) récupérés
 - ☐ Apple Sign In activé sur l'App ID
 - ☐ Services ID Apple créé + Key ID + Team ID notés
 - ☐ Fichier `.p8` téléchargé et transmis au backend (en sécurité)
 - ☐ Variables `EXPO_PUBLIC_GOOGLE_*` remplies dans `apps/mobile/.env`
 - ☐ EAS dev build iOS installée sur l'iPhone de test
 - ☐ EAS dev build Android installée sur le smartphone Android de test
 - ☐ Backend `/auth/social` remplacé (501 → vérification JWKS)
 - ☐ Test bout-en-bout réussi : Google iPhone, Google Android, Apple iPhone
-

SGI — Infinity International Facilities Management UAE

Document généré pour faciliter la mise en production du social login.

En cas de blocage, vérifier que les `EXPO_PUBLIC_GOOGLE_*_CLIENT_ID` sont bien chargés (`npx expo start` imprime `env: load .env`).